0.75.1. Arrivé en (0, 0), le A devient B — de base, ailleurs

Le 0.75 avec un autre départ : le A part de (10, 8), et devient toujours B en (0, 0). Il faut aller à gauche, puis monter.

1. Ce qu'il faut comprendre

C'est le 0.75, avec un seul changement : le **départ**. Le A part de **(10, 8)**, au milieu de l'écran, au lieu de (10, 0).

La case de transformation ne change pas : (0, 0), le coin en haut à gauche. La condition est la même : `if (x == 0 && y == 0)`.

Le chemin est plus long : la flèche **gauche** amène le A en (0, 8), au bord gauche. Là, `x == 0` est vrai, mais `y == 0` est faux (y vaut 8) : il reste un A. Puis la flèche **haut** le fait monter : 7, 6… et en (0, 0), les deux sont vrais. Il devient B.

C'est ce que montre `&&` : arriver dans la bonne colonne ne suffit pas, il faut aussi la bonne ligne. Regarde les nombres en bas : `000 008` au bord gauche, encore un A ; `000 000` au coin, un B.

C'est la version de base : une seule lettre, un autre départ. Le 0.75.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <deplace_croix>` → 0.66.3. La fonction deplace_croix() — suivre la croix, case par case
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <nombre>` → 0.8.3. La fonction nombre() — écrire un nombre

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **opérateurs logiques** — && « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

```
#include <deplace_croix>   // une tuile qui suit la croix, case par case
```

Demande à la console « deplace_croix » (ce que c'est : une tuile qui suit la croix, case par case). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui suit la croix, case par case ».)

`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <nombre>  // écrit un nombre en chiffres`	Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un nombre en chiffres ».)
`uint8_t x = 10;  // le changement : le départ, (10, 8), au milieu`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 10. (Le commentaire le dit : « le changement : le départ, (10, 8), au milieu ».)
`uint8_t y = 8;`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 8.
`uint8_t lettre = 0;  // 0 = A, 1 = B`	Une variable : une boîte appelée lettre, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « 0 = A, 1 = B ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`deplace_croix(x, y, ALPHABET[lettre], 250);`	Appel de la fonction deplace_croix de la console, avec les arguments x, y, ALPHABET[lettre], 250. Ce qu'elle fait : la lettre suit la croix, case par case : à appeler dans la boucle, après image(). vitesse : le temps entre deux pas, en ms écrites en clair (250 = 4 cases par seconde, 100 = 10). Reste dans l'écran, efface seulement si elle a bougé (pas de clignotement). Seule sur sa ligne : deplace_croix(x, y, ALPHABET[0], 250); range la place dans x et y.
`nombre(0, 17, x);  // la position, en direct`	Appel de la fonction nombre de la console, avec les arguments 0, 17, x. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la position, en direct ».)
`nombre(4, 17, y);`	Appel de la fonction nombre de la console, avec les arguments 4, 17, y. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand.
`// La même case qu'au 0.75 : (0, 0), le coin en haut à gauche.`	Un commentaire : une phrase pour celui qui lit (« La même case qu'au 0.75 : (0, 0), le coin en haut à gauche. »). La console l'ignore.
`//  en (0, 8) : x == 0 vrai, y == 0 faux → encore un A (il faut monter)`	Un commentaire : une phrase pour celui qui lit (« en (0, 8) : x == 0 vrai, y == 0 faux → encore un A (il faut monter) »). La console l'ignore.
`//  en (0, 0) : les deux vrais  → le A devient B`	Un commentaire : une phrase pour celui qui lit (« en (0, 0) : les deux vrais → le A devient B »). La console l'ignore.
`// Sur une ligne :  if (x == 0 && y == 0) lettre = 1;`	Un commentaire : une phrase pour celui qui lit (« Sur une ligne : if (x == 0 && y == 0) lettre = 1; »). La console l'ignore.
`// Avec des accolades, la même chose :`	Un commentaire : une phrase pour celui qui lit (« Avec des accolades, la même chose : »). La console l'ignore.

`if (x == 0 && y == 0) {`	Une décision : si x est égal à 0 et y est égal à 0, on fait ce qui est entre les accolades. Sinon, on passe.
`lettre = 1;`	lettre prend une nouvelle valeur : 1.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1255 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage         76 o  ← la console, à l'allumage  — la mise en route de la console
  deplace_croix()  217 o  ← main()  — une fonction fournie par le langage
  main()            70 o  ← démarrage  — ta fonction
  VBlank             8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage     28 o  ← image()  — attend l'image suivante
  AttendreVBlank    16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces     12 o  ← EffacerCases, EcrireNombre  — attend de pouvoir écrire une case
  AdresseCase       10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases      12 o  ← effacer()  — efface des cases
  EcrireNombre      33 o  ← nombre()  — écrit un nombre en chiffres
  CopierTuiles      18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte      14 o  ← démarrage  — vide les deux cartes de l'écran
  LireManette       37 o  ← bouton()  — lit les huit boutons
  Tuiles           704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
  bouton()        × 4    20 o sur place, dans deplace_croix()  + LireManette (37 o)
  nombre()        × 2    20 o sur place, dans main()  + EcrireNombre (33 o)
  effacer()       × 1    15 o sur place, dans deplace_croix()  + AdresseCase (10 o), EffacerCases (12 o)
  poser()         × 1    13 o sur place, dans deplace_croix()  + AdresseCase (10 o)
  image()         × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  images()        × 1     2 o sur place, dans deplace_croix()
Tes fonctions appelées :
  deplace_croix() × 1    21 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix>   // une tuile qui suit la croix, case par case
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <nombre>          // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 10;       // le changement : le départ, (10, 8), au milieu
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;   // 0 = A, 1 = B
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

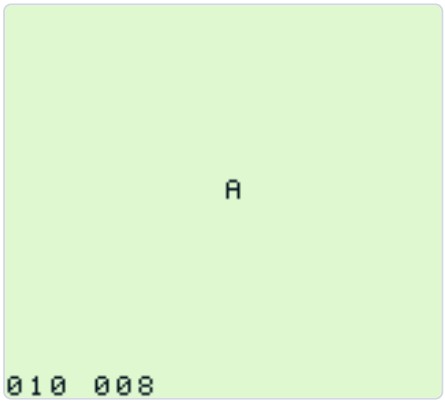
```
int main() {
  while (true) {
    image();
    deplace_croix(x, y, ALPHABET[lettre], 250);
    nombre(0, 17, x);                    // la position, en direct
    nombre(4, 17, y);

    // La même case qu'au 0.75 : (0, 0), le coin en haut à gauche.
    //
    //   en (0, 8) : x == 0 vrai, y == 0 faux → encore un A (il faut monter)
    //   en (0, 0) : les deux vrais           → le A devient B
    // Sur une ligne :  if (x == 0 && y == 0) lettre = 1;
    // Avec des accolades, la même chose :
    if (x == 0 && y == 0) {
      lettre = 1;
    }
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un A au milieu de l'écran ; la flèche gauche l'amène au bord (0, 8), encore un A ; la flèche haut le fait monter jusqu'au coin (0, 0), où il devient B.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ au bord (0, 8), c’est encore un A

✓ en (0, 0), il est devenu B

✓ la position s’affiche : 000 000 (« 000 000 »)

Ensuite : **0.75.2. Arrivé en (0, 0), le A devient B — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++